

Multi-Agent Systems and Agentic AI

Practical Examination

This examination has two parts. **Part I** covers GRPO finetuning of a language model and has two components: a *practical* component (sections I.1–I.3 — a one-week project carried out by teams of 2–3 students on a dedicated Google Cloud TPU, v6e-1, one per team) and a *theory* component (section I.4 — written questions, completed individually). The report on the practical component is individual — see the submission rules below. **Part II** is completed individually: you read, diagram, and critique an adaptive-planning implementation.

Submission. You submit a *single* PDF report covering both parts. Page limits, counted excluding references and appendix:

- Part I, sections I.1–I.3 (practical write-up): at most **3 pages**.
- Part I, section I.4 (theory): no page limit, but answers must be clear and concise — verbosity will be penalised.
- Part II: at most **2 pages**, excluding the workflow diagram.

Unless stated otherwise, these page limits assume the report is typeset on **A4 paper, 11 pt body font, single column, margins of at least 2 cm, and single line spacing** — the format of this question paper. Submissions that shrink the font, margins, or line spacing to evade a limit will be treated as over-length. **The report is individual: every student submits their own PDF covering both parts.** Sections I.1–I.3 describe a team project — the TPU is shared, the runs are shared, the code repository is shared — but each team member writes up the practical component in their own words, in their own PDF. Two students from the same team may legitimately emphasise different runs, plots, or interpretations of the same data; what they may not do is submit the same prose. State your team members explicitly on the title page so that the marker can identify which experiments are jointly owned. Section I.4 (theory) and Part II are strictly individual. Appendices do not count towards the page limit but must not contain material essential to the marking.

Section I.4 must be typeset in L^AT_EX. All section I.4 (theory) answers must be written in L^AT_EX — no scanned handwriting, no photographed whiteboards, no Word equations. Use `amsmath` environments (`align`, `equation`, ...) and number any equation you refer back to. The rest of the report should be written in L^AT_EX as well, so that the whole submission is a single coherent document.

Marks weighting. The exam carries **120 marks** in total: Part I has a practical component (sections I.1–I.3, marked out of 50) and a theory component (section I.4, marked out of 40); Part II is marked out of 30. The final mark is simply the total, expressed as a percentage,

$$\text{final} = \frac{(\text{I.1–I.3}) + \text{I.4} + \text{II}}{120}.$$

The marks are allocated so that the practical / applied work — sections I.1–I.3 together with Part II, which covers roughly two thirds of the lecture course — counts for two thirds of the final mark (80/120), and the theory (section I.4) for one third (40/120). The mark counts shown on each header are the actual marks: no separate weighting step is applied.

Code and submission of code. You may use GitHub while developing Part I — it is convenient for sharing within a team — but, as for previous coursework, all code must ultimately be ported to your own **GitLab** repository, and it is the GitLab repository that constitutes the submission. Link the GitLab repository from the report. All training and evaluation logs (W&B, TensorBoard, JSONL, or equivalent) must be included or made accessible from it.

Code accountability. Use of AI assistance while writing code is expected and permitted. You nonetheless remain fully responsible for every line you submit: in the viva you must be able to explain any line of your code on request.

Links. Every URL that appears in the report — GitLab repository, W&B run, dashboard, dataset card, paper — must be a *clickable* hyperlink in the submitted PDF (use `\href{...}{...}` or `\url{...}` in $\text{\LaTeX}$; equivalent in other tools). Anything cited that lives outside the report (datasets, baselines, external code, papers) must come with a clickable URL the marker can follow without retyping.

Part I — GRPO finetuning of Gemma 3 on a TPU (90 marks, $\frac{3}{4}$ of final)

Format. Teams of 2–3 students. Each team is given access to a **v6e-1** TPU for a one-week window: from **Monday 8 June 2026** until **Monday 15 June 2026**. Setup of the TPU (project access, quota, SSH keys, base image) will be performed for you, and step-by-step guidelines for connecting to and using the TPU will be provided in the [tpu-2026](#) repository. All TPUs are started simultaneously, so the window is the same for every team regardless of when each team first logs in. A single full GRPO training run on the baseline takes roughly 5–9 hours on **v6e-1**, so the window typically accommodates only a handful of complete runs — plan accordingly (debug at small scale first, decide which sweep to run, then commit the wall-clock to it).

Starting point. Each team is given the [tpu-2026](#) GitHub repository as its baseline. The repository contains a complete, working GRPO finetuning pipeline for `google/gemma-3-1b-it` on the GSM8K math word-problem dataset, built on Tunix / JAX with LoRA adapters and a programmatic (non-learned) reward. The salient pieces are:

- [scripts/train.py](#) — GRPO training entry point (Tunix `GRPOLearner`, Orbax checkpointing, W&B logging).
- [scripts/rewards.py](#) — four reward functions (exact and approximate format matching, exact / close numeric answer, and a number-extraction fallback); the rollout reward is their sum.
- [scripts/config.py](#) — the single source of truth for every hyperparameter you might tune (LoRA rank, K , β , ε , learning rate, batch sizes, generation length, mesh shape, ...).
- [scripts/evaluate.py](#) — held-out evaluation against the GSM8K test split.
- [scripts/data.py](#), [scripts/model.py](#), [scripts/chat.py](#) — data loading, model / tokenizer construction, and an interactive sampler.
- [tunix.ipynb](#) — the original notebook the scripts were decomposed from; useful as a tutorial.

Background reading. The baseline finetunes only LoRA (Low-Rank Adaptation) adapters on top of the frozen base model. Appendix A is a one-page reminder of the mechanism; if you are not already comfortable with LoRA, start there and then read the original paper, Hu et al. (2021), arxiv.org/abs/2106.09685.

Goal. At minimum: (i) get the baseline running on your **v6e-1**, (ii) understand what every non-trivial piece of it does, and (iii) attempt at least one *principled* improvement over the baseline, chosen and justified using the theory developed in the lectures. You are not expected to beat the baseline by a large margin — a small, well-controlled improvement that you can *explain* is worth more than a large but unexplained jump.

Deliverable. Sections I.1–I.3 of *your own* report contain plots and tables that compare your team’s run(s) to the baseline, plus a written discussion of your design choices — written by you, not copy-pasted from a team-mate. A clickable link to the team’s GitLab repository (see the submission rules) with all changes committed must appear in the report.

I.1 Reproducing the baseline (10 marks)

Run `scripts/train.py` end-to-end on your **v6e-1** with the default `scripts/config.py`, then run `scripts/evaluate.py` on the resulting checkpoint. In your report:

- (i) State the exact commit you ran, the wall-clock time of the run, and the number of GRPO steps actually completed.
- (ii) Report baseline GSM8K accuracy of (a) the *base* model `gemma-3-1b-it` and (b) the LoRA-finetuned checkpoint, on a clearly-stated held-out split with a fixed seed.
- (iii) Plot the baseline training curve (mean reward $\bar{r}$ vs. GRPO step), and the KL divergence $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ over training.

If anything in the baseline did not run as-shipped on your hardware, document the fix in a single “baseline patches” section.

I.2 Understanding the baseline (10 marks)

In the report, in at most one page, answer the following *in your own words*, mapping the baseline onto the formal objects defined in the lectures:

- (a) What plays the role of π_θ , π_{old} and π_{ref} in the LoRA setup, and which of them carries trainable parameters?
- (b) What is the group size K in `scripts/config.py`, and which advantage estimator does Tunix use? State whether the baseline computes the standard or the leave-one-out advantage.
- (c) Identify, in `scripts/rewards.py`, which terms are shaping rewards and which is the “true” correctness reward. Explain, in terms of within-group reward variance σ_r , what would happen early in training with the correctness reward alone.
- (d) Locate where the clipped (PPO-style) surrogate is implemented in Tunix, and state what the clip range ε corresponds to in `scripts/config.py`. Note any deviation from the form given in the lectures (e.g. a per-token rather than per-sequence ratio).

I.3 Improving on the baseline (30 marks)

Make at least one *principled* modification to the baseline and run a controlled comparison against it. The modification must be motivated in writing, grounded in the theory developed in the lectures. Acceptable directions include, but are not limited to:

- **Algorithmic.** Replace the standard group-mean baseline with the leave-one-out advantage; sweep the group size K and check how the variance of $\hat{A}_i$ scales empirically; change β / ε and report KL drift; switch the reference-KL penalty on or off.

- **Reward.** Modify `scripts/rewards.py` — e.g. remove a shaping term, change its weight, add a length penalty to mitigate response-length blowup, or replace approximate format matching with strict matching.
- **Data / curriculum.** Filter or re-order GSM8K, or mix in a second math source; explain how this changes the distribution of within-group reward variance σ_r .
- **Optimisation / capacity.** Change LoRA rank, learning rate, schedule, or generation length, and explain the trade-off in terms of KL budget and effective sample size.

The comparison must be *controlled*: same data, same eval split, same total compute (or normalised per step). We expect a typical report to cover **at least three complete training runs** in total — baseline plus two further runs, whether different modifications or different seeds of the same modification. Fewer is acceptable only with an explicit justification (e.g. TPU failure documented in the “baseline patches” section).

In your report, include:

- (a) A table comparing the base model, the baseline GRPO checkpoint, and your improved checkpoint(s) on the held-out GSM8K split, with seeds and a measure of uncertainty (second seed or bootstrap CI over the eval prompts).
- (b) Training curves of *mean reward* and *KL* for the baseline *and* your variants on a single set of axes.
- (c) One diagnostic plot addressing a known GRPO failure mode — for example the distribution of $\hat{A}_i$ over training, the fraction of degenerate groups in which all K rewards are equal, the response length / entropy of π_θ over time, or $\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ vs. task accuracy.
- (d) A discussion connecting your empirical findings to the theory developed in the lectures, including any prediction you have *contradicted*.

Honest negative results — “we tried X, the theory predicted Y, we observed Z” — are explicitly in scope and will be marked on the quality of the diagnosis, not on whether the run won.

I.4 GRPO theory — written questions (40 marks)

Answer all questions. This is the theory component of Part I, completed *individually* and typeset in L^AT_EX (see the submission rules). It accounts for one third of the final mark (40 of the 120 marks).

1. Fix a prompt q and $K \geq 2$ responses $a_1, \dots, a_K$, which are *treated as fixed throughout this question*. Let $r_1, \dots, r_K$ be i.i.d. random rewards with mean μ and variance σ^2 (so the only randomness in this question is in the rewards). Define the group sample mean and the normalised advantage

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i, \quad \hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r},$$

treating $\sigma_r = \sigma$ as a known constant for the purpose of variance computations. Let

$$h_i = \nabla_\theta \log \pi_\theta(a_i | q) \in \mathbb{R}^d$$

denote the (deterministic) score vectors of a parametric policy π_θ , and define

$$X_i = h_i \hat{A}_i, \quad \hat{g} = \frac{1}{K} \sum_{i=1}^K X_i \in \mathbb{R}^d.$$

- (i) Compute $\mathbb{E}[X_i]$ and hence $\mathbb{E}[\hat{g}]$. Explain in up to two sentences why $\mathbb{E}[\hat{g}] = 0$ here does *not* mean the underlying policy gradient vanishes. (2)
- (ii) The X_i are not independent because they share a common random term. Identify that term explicitly. Then compute $\text{Cov}(X_i, X_j)$ for $i \neq j$, expressing the result in terms of the outer product $h_i h_j^\top$ and the moments of r . State the sign of the result and explain intuitively why this sign arises. (6)
- (iii) Compute the covariance matrix $\text{Var}(\hat{g}) \in \mathbb{R}^{d \times d}$. Compare with the naive expectation that an i.i.d. estimator built from K copies of X_1 would give $\text{Var}(\hat{g}) = \frac{1}{K} \text{Var}(X_1)$. Use the discrepancy to define an *effective sample size* K_{eff} , give an explicit expression for it in terms of $\text{Var}(X_1)$, and comment briefly on its behaviour for large K and on what happens when all the h_i are aligned in a single direction. (7)

2. Let $\mathcal{A}$ be a finite (or measurable) action space. A *policy* is a probability distribution on $\mathcal{A}$. Let $\pi_t : \mathcal{A} \rightarrow [0, 1]$ denote the policy at iteration t (so $\sum_a \pi_t(a) = 1$, read as an integral if $\mathcal{A}$ is continuous), and let $A_t : \mathcal{A} \rightarrow \mathbb{R}$ be a fixed function called the *advantage at iteration t* . For two policies p, q , the KL divergence is

$$\text{KL}(p||q) = \sum_a p(a) \log \frac{p(a)}{q(a)}, \quad (\text{with } 0 \log 0 = 0).$$

Consider the regularised update

$$\pi_{t+1} = \arg \max_{\pi} \left\{ \mathbb{E}_{a \sim \pi}[A_t(a)] - \frac{1}{\eta} \text{KL}(\pi || \pi_t) \right\}, \quad \eta > 0, \quad (\dagger)$$

where the maximum is over all probability distributions on $\mathcal{A}$.

- (i) Find the unique optimiser of $(\dagger)$ and explain the role of η as a step-size parameter. (3)
- (ii) For a policy π , define the scalar performance $J(\pi) = \mathbb{E}_{a \sim \pi}[A^{\pi_t}(a)]$, where A^{π_t} is the *true* advantage function of the current iterate π_t . A sequence of policies $\{\pi_t\}$ is said to satisfy *monotone improvement* if $J(\pi_{t+1}) \geq J(\pi_t)$ for every t . Show that, when the advantage used in $(\dagger)$ is exact ($A_t = A^{\pi_t}$), the update $(\dagger)$ guarantees monotone improvement. Identify one practical condition under which this guarantee fails when GRPO is applied to a neural-network policy. (5)
- (iii) In practice, GRPO replaces the KL penalty in $(\dagger)$ by a clipped surrogate of PPO type. Let $\pi_{\text{old}} = \pi_t$ and let $a_t \sim \pi_{\text{old}}(\cdot | q)$ be a sample for a prompt q . Define the probability ratio and the clipped objective

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | q)}{\pi_{\text{old}}(a_t | q)},$$

$$J^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right],$$

where $\hat{A}_t$ is an estimate of $A_t(a_t)$ and $\varepsilon \in (0, 1)$ is the clip parameter. Describe the *trust region* on the policy update that the clip induces, and contrast it with the trust region defined by the single joint constraint $\text{KL}(\pi_\theta(\cdot | q) || \pi_{\text{old}}(\cdot | q)) \leq \delta$ implicit in $(\dagger)$. State at least one qualitative difference. (2)

3. Fix a prompt q and let $\pi_{\text{old}}(\cdot | q)$ be a fixed sampling policy (the “old” policy) over responses a . Let $R(q, a) \in \mathbb{R}$ be a deterministic scalar reward, and define the value of any policy π as

$$V^\pi(q) = \mathbb{E}_{a \sim \pi(\cdot | q)} [R(q, a)].$$

Standard GRPO samples $a_1, \dots, a_K \stackrel{\text{iid}}{\sim} \pi_{\text{old}}(\cdot | q)$, sets $r_i = R(q, a_i)$, and forms the estimator

$$\hat{g}_{\text{GRPO}} = \frac{1}{K} \sum_{i=1}^K \nabla_\theta \log \pi_\theta(a_i | q) \cdot \hat{A}_i, \quad \hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r},$$

where $\bar{r} = \frac{1}{K} \sum_i r_i$ and σ_r is the empirical reward standard deviation.

Consider instead drawing a_i from the *mixture proposal*

$$\pi_{\text{mix}}(a | q) = (1 - \alpha) \pi_{\text{old}}(a | q) + \alpha \pi_{\text{unif}}(a | q), \quad \alpha \in [0, 1),$$

where π_{unif} is the uniform distribution over responses.

- (i) The true policy gradient is an expectation under π_{old} . Derive an importance-corrected gradient estimator $\hat{g}_{\text{mix}}$ that uses samples drawn from π_{mix} instead of π_{old} , and give the explicit form of the importance weight w_i associated with each sample. (3)
- (ii) Verify that $\hat{g}_{\text{mix}} \rightarrow \hat{g}_{\text{GRPO}}$ (with the same realised actions a_i) as $\alpha \rightarrow 0$. (1)
- (iii) Under mixture sampling, the group mean $\bar{r}$ is now an estimate of $V^{\pi_{\text{mix}}}(q)$, not $V^{\pi_{\text{old}}}(q)$.
 - a. Express $V^{\pi_{\text{mix}}}(q)$ in terms of $V^{\pi_{\text{old}}}(q)$ and $V^{\pi_{\text{unif}}}(q)$. (1)
 - b. Determine whether the importance weights w_i alone correct for this baseline shift, and derive the correct form of the reweighted advantage that should appear in $\hat{g}_{\text{mix}}$ to keep it unbiased for the policy gradient under π_{old} . (2)
- (iv) Compare the variance of $\hat{g}_{\text{mix}}$ to that of $\hat{g}_{\text{GRPO}}$ for the same K . Identify one condition on R under which mixing with π_{unif} *increases* the variance of the advantage estimates, and a different condition under which it *decreases* it. Give a brief mathematical argument in each case. (4)
- (v) Suppose responses are sequences of T tokens. To keep the analysis concrete, assume the per-token importance ratio

$$\frac{\pi_{\text{old}}(a_{i,t} | q, a_{i,<t})}{\pi_{\text{mix}}(a_{i,t} | q, a_{i,<t})} = c$$

takes the same value $c \in (0, 1)$ at every step, so that the full-response importance weight is

$$w_i = c^T.$$

- a. Describe the behaviour of w_i as $T \rightarrow \infty$, and explain in one sentence why a value $c < 1$ is the typical situation when $\alpha > 0$. (2)
- b. This phenomenon is called *weight collapse*. Explain briefly why it makes $\hat{g}_{\text{mix}}$ unreliable for long responses, and propose one concrete fix — to π_{mix} or to the weighting scheme — stating what it trades off. (2)

Part II — Adaptive planning in `cmbagent_lg` (individual, 30 raw marks, $\frac{1}{4}$ of final)

Format. Individual. This part is deliberately lighter than Part I: you are not asked to build a system, but to *read, diagram, and critique* one.

Background. In the lectures we contrasted *open-loop* planning — produce a plan once, then execute it unchanged — with *adaptive* (closed-loop) planning, in which the plan can be rewritten *while it is being executed*, in response to what has actually happened. `cmbagent_lg` ships a LangGraph implementation of adaptive planning: a planner / plan-reviewer loop produces an initial plan, a control loop executes it step by step, and after each step an adaptive-review mechanism may rewrite the remaining steps before execution continues. The implementation to study lives on the `adaptive-planning` branch of `cmbagent_lg`, in the `cmbagent_lg/adaptive_planning` module (it composes the existing `planning/` and `deep_research/` modules). This branch is frozen — it will not change — so study it at:

https://github.com/borisbolliet/cmbagent_lg/tree/adaptive-planning

Read that implementation in full — the graph, its nodes, and the state it threads — before answering.

II.1 Workflow diagram (12 marks)

Produce a clear diagram of the adaptive-planning workflow as implemented on the [branch linked above](#) — for instance the LangGraph graph: its nodes, its edges, the conditional routing, and the state that flows between nodes. Accompany the diagram with a short written walkthrough (at most half a page) that makes explicit: what triggers a plan revision, what part of the plan may change, what is held fixed across a revision, and how the loop is guaranteed to terminate.

II.2 Problems (9 marks)

Identify and explain the most significant problems, limitations, or failure modes of the implementation. Each problem must be argued, not merely asserted, and grounded in the lecture material on planning and closed-loop control.

II.3 Proposed improvements (9 marks)

For the problems you raised in II.2, propose concrete, justified improvements. For each, state what it changes, why it addresses the problem, and what it trades off (extra cost, added latency, new failure modes). You are not required to implement them, but each proposal must be specific enough that someone could.

Deliverable. The Part II section of the report contains the workflow diagram, the critique, and the proposed improvements. Include a clickable link to the exact `cmbagent_lg` commit (on the `adaptive-planning` branch) you studied. The workflow diagram does not count towards the 2-page limit.

Appendix A — LoRA in one page

This appendix is a brief reminder of the mechanism the baseline relies on; it is not examinable on its own. For the full treatment see Hu et al. (2021), arxiv.org/abs/2106.09685.

The reparameterisation. Let $W_0 \in \mathbb{R}^{d \times k}$ be a frozen pretrained weight matrix (e.g. an attention or MLP projection of `gemma-3-1b-it`). *Low-Rank Adaptation* (LoRA) leaves W_0 untouched and adds a trainable low-rank update,

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k},$$

with rank $r \ll \min(d, k)$ and a fixed scaling α/r . Only A and B are trained; W_0 stays frozen. The parameter count of the adapter is $r(d + k)$, typically a fraction of a percent of $|W_0| = dk$, which is what makes finetuning a 1B model feasible on a single `v6e-1`.

Initialisation. A is initialised with small random (Gaussian) entries and B is initialised to *zero*, so that $\Delta W = 0$ at the start of training and the adapted model is *exactly* the base model. The forward pass adds the two paths, $h = W_0 x + \frac{\alpha}{r} B(Ax)$, so no information is lost and the update grows smoothly away from the base.

In the GRPO baseline the three policies share the frozen W_0 and differ only in their adapters: this is why π_{ref} (the base, $B = 0$) is recovered exactly at initialisation and costs nothing to keep around for the reference-KL penalty, while π_θ and π_{old} are the same network with different adapter snapshots (see I.2(a)). The LoRA rank r is set in `scripts/config.py`; raising it increases adapter capacity (and the effective step the KL budget must absorb) at the cost of more trainable parameters — one of the optimisation / capacity knobs available in I.3.